Security and safety practices

Keep policy text contained and the system safe.

For: Everyone

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

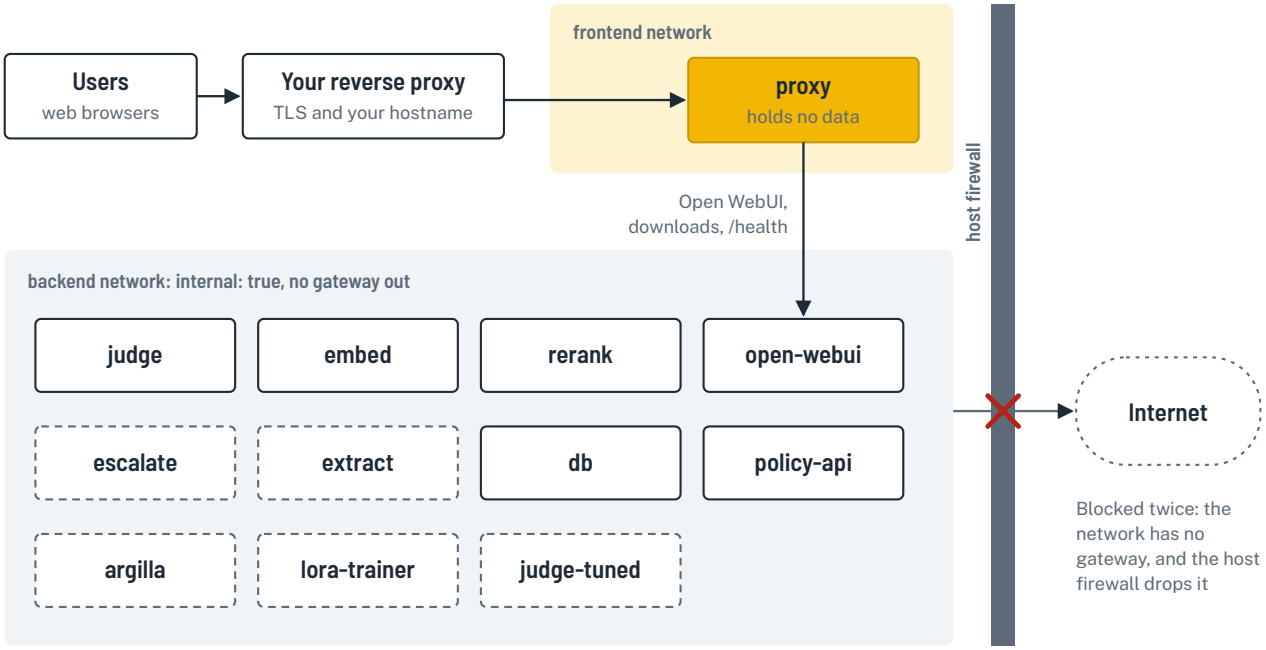
Two things must hold: **policy text never leaves the contained environment**, and **no one acts on an automated decision without a person checking it**. This page explains how the system enforces both, and what you still have to do.

Threats and controls

Risk	Control in this repo	Your job
Policy text leaves the environment	Every service holding text is on the `internal: true` network; models served locally; `HF_HUB_OFFLINE`, telemetry off	Run `check_containment.sh`; block egress at the host firewall too
Unvetted or tampered model weights	Weights mounted read-only; SHA-256 manifest	Record hashes at download; check before each start; approved-model list
Someone calls the API directly	Bearer key on every route; proxy exposes only downloads and `/health`	Keep `.env` at `chmod 600`; rotate keys
A malicious web page uses a browser to reach the API (DNS rebinding)	`TrustedHostMiddleware` refuses unknown `Host` headers	Keep `ALLOWED_HOSTS` short
A download link is shared or guessed	HMAC-signed links that expire after 72 h; run IDs are random UUIDs	Shorten `LINK_TTL_HOURS` if needed
Instructions hidden inside a policy steer the model ("ignore the above...")	Policy text is fenced as data in every prompt, and fence markers inside it are replaced so it can't break out; the judge returns only a fixed JSON schema; a "yes" whose quoted evidence isn't really in both texts is downgraded to Maybe for a person to check; the chat model is told never to follow text in documents	Investigate verdicts whose reasons look odd
Policy text becomes a live Excel formula	Cells that would be formulas are forced to text	Nothing
Document or model text adds a link or image to the chat answer (an image URL could carry data out)	Markdown characters in every document and model value are escaped; only the download link is live	Keep Open WebUI on the internal network; block egress so a stray image can't load

Risk	Control in this repo	Your job
A small upload expands into a huge one (zip bomb, giant PDF) or a crafted line slows parsing	Uploads capped at `MAX_UPLOAD_MB`; .docx refused if it unpacks past 50 MB; PDFs capped at 1,000 pages; text capped at 5 M characters; each upload is parsed in its own process (1 GB memory cap, killed after 60 s, at most 2 at once); heading regex can't backtrack	Nothing
The model is wrong	Evidence quotes and reasons on every row; "Maybe" for uncertainty; full audit trail	Keep a person approving every change (below)
Lost history	Every model decision stored in `judgements` with model name and prompt hash	Back up the database (operations.md)

Network containment



Two barriers keep policy text in: a network with no gateway, and your host firewall.

- `backend` is a Docker network with `internal: true`, so its containers have no gateway out.
- Only `proxy` joins `frontend`. nginx serves Open WebUI and only `/exports/<run-id>.xlsx` and `/health` from policy-api. All other API routes return 404 from outside.
- **Don't rely on Docker settings alone.** Also block outbound traffic from the host at your firewall (for example, rules in the `DOCKER-USER` chain on Linux), and keep the server on a network segment without internet access.

Proving it

```
scripts/check_containment.sh
```

For each running service, the script first connects to an internal service. That's the positive control: it proves the probe itself works. Then it tries to reach the internet, and that must fail. The script also fails if it checked nothing. Run it after setup, after every upgrade, and after any network change. Keep the output with your change records.

During development, a network setting meant to block outbound traffic ("no outbound NAT") was ignored by Docker Desktop. The script caught it. That's why containment relies on `internal: true` networks, not driver options.

Secrets

- Every secret lives in `.env` (git-ignored). Generate each with `openssl rand -hex 32`. policy-api refuses to start if its keys are shorter than 32 characters.
- `chmod 600 .env`. Only administrators can read the server.
- **Rotate** by changing the value and restarting the affected services:
- `POLICY_API_KEY`: restart `policy-api` and update the tool's Valve in Open WebUI
- `LINK_SIGNING_KEY`: restart `policy-api`; existing download links stop working
- `LLM_API_KEY`: restart the model services, `policy-api` and `open-webui`
- Never paste secrets into chats, tickets or a wiki.

Models and supply chain

- Use only models on your organisation's approved list. All defaults are US-developed and published under permissive licenses. Check each model card and license before use.
- Download weights outside the environment, record hashes (`scripts/verify_models.sh record`), carry them in, and check them (`verify_models.sh check`) before every start and upgrade.
- Pin every image tag in `.env`. Never use `latest`. Build the policy-api image from a pinned dependency set or your internal mirror.
- vLLM's `--trust-remote-code` (needed for Inkling) runs Python shipped with the model. Review that code and verify the hashes before enabling `escalate`.

Access

- Open WebUI accounts are created by an admin. Sign-up is off after the first admin.
- Everyone with an account can query every group. If some groups must be restricted, run a separate stack per audience, or add per-user group filtering to the tool before go-live.
- The tool sends the asking user's email or name as `requested_by`, which is stored on the run. Tell users their questions are logged.

Human oversight

The system **finds and explains**. It does not approve changes.

- Every "Yes" and "Maybe" goes to a named person, who records a decision (the Review decision column, later the review queue).
- "Maybe" never means "fine". Not-affected policies are listed, with the reason, so reviewers can spot-check them.
- Don't let the tool's output become the record of decision. The decision belongs in your document control system.
- Measure accuracy on real past changes before launch and every quarter (evaluation.md).

Data retention

Uploaded files, runs, verdicts and Excel files stay in the database and the `exports` volume until you delete them. Agree on a retention period with records management. To purge runs older than a set age:

```
DELETE FROM runs WHERE created_at < now() - interval '2 years';   -- cascades to changes and
judgements
DELETE FROM uploads u WHERE NOT EXISTS (SELECT 1 FROM runs r WHERE r.upload_id = u.id)
                        AND u.created_at < now() - interval '2 years';
```

Then delete the matching `<run-id>.xlsx` files from the `exports` volume.

If something goes wrong

Situation	Do this
`check_containment.sh` fails	Stop `open-webui` and `policy-api` (`docker compose stop`). Fix the network. Re-run the check. Record what happened
A secret may have leaked	Rotate it now (see Secrets). Check the proxy logs for unusual downloads
A wrong result was acted on	Find the run ID on the Excel Summary sheet; pull its `judgements`; add the case to the evaluation set
A policy contains text that tries to instruct the model	Keep the document for analysis; check the verdicts for that policy by hand

Go-live checklist

- [ ] `check_containment.sh` passes and the output is filed
- [ ] Host firewall blocks outbound traffic from the server
- [ ] `.env` secrets generated fresh, file is `chmod 600`, not in git
- [ ] Model hashes verified; models on the approved list
- [ ] Image tags pinned; none is `latest`
- [ ] Open WebUI sign-up turned off; users created by an admin
- [ ] TLS on the reverse proxy in front of `proxy`; `PUBLIC_BASE_URL` uses `https://`
- [ ] Evaluation recall meets the agreed target on a hold-out set
- [ ] Review process for Yes/Maybe agreed with the policy owners
- [ ] Database backups running and restore tested

- [ ] Retention period agreed and written down